

CRITERIOS PARA ELEGIR UNA ARQUITECTURA

J. S. Morales López

7690-08-6790 Universidad Mariano Gálvez

Seminario de tecnologías de información

jmorales115@miung.edu.gt

Repositorio en GitHub - Sofia Morales

Resumen

El presente artículo analiza los principales criterios que deben considerarse para seleccionar la arquitectura de una aplicación. Esta decisión no depende únicamente de si el software será web, móvil o de escritorio, sino también de su complejidad, cantidad de usuarios, volumen de información, integraciones, disponibilidad requerida, presupuesto y experiencia del equipo de desarrollo. Se describen arquitecturas como la monolítica, en capas, de microservicios, orientada a eventos y sin servidor. Asimismo, se relacionan estas alternativas con aplicaciones pequeñas, sistemas empresariales, plataformas de gran escala, soluciones de Internet de las cosas y aplicaciones con demanda variable. El análisis demuestra que cada arquitectura presenta ventajas, limitaciones y costos técnicos que deben evaluarse antes de iniciar el desarrollo. Una arquitectura sencilla puede ser suficiente para un proyecto pequeño, mientras que un sistema complejo podría requerir componentes independientes y escalables. Se concluye que la arquitectura adecuada es aquella que satisface los requisitos actuales del proyecto, permite su mantenimiento y puede evolucionar sin introducir una complejidad innecesaria.

Palabras clave: arquitectura de software, aplicaciones, microservicios, escalabilidad y requisitos.

Desarrollo del tema

La arquitectura de software representa la organización general de una aplicación. Describe sus componentes principales, las responsabilidades de cada uno y la manera en que intercambian información. Entre estos componentes pueden encontrarse la interfaz de usuario, las reglas de negocio, las bases de datos, los servicios y las integraciones con sistemas externos.

La selección de una arquitectura influye en el rendimiento, la seguridad, la mantenibilidad y la capacidad de crecimiento del sistema. Por esta razón, no debe seleccionarse solamente porque sea moderna o porque haya sido utilizada por una empresa reconocida. Según Kazman et al. (2023), las decisiones arquitectónicas deben relacionarse con las necesidades del sistema y con sus atributos de calidad. Esto significa que una arquitectura apropiada para una plataforma de comercio electrónico de gran tamaño puede resultar innecesaria para una aplicación académica utilizada por pocas personas.

Criterios para elegir una arquitectura

Antes de seleccionar una arquitectura, se deben identificar los requisitos funcionales y no funcionales. Los requisitos funcionales establecen lo que debe realizar la aplicación, mientras que los no funcionales determinan condiciones como seguridad, rendimiento, disponibilidad, facilidad de man-

tenimiento y escalabilidad.

Los principales criterios de selección son los siguientes:

- **Tamaño y complejidad:** se debe determinar cuántos módulos, procesos y reglas de negocio tendrá la aplicación.
- **Cantidad de usuarios:** permite estimar la carga de trabajo que deberá soportar el sistema.
- **Volumen de información:** ayuda a definir las necesidades de almacenamiento, procesamiento y consulta de datos.
- **Disponibilidad:** establece si la aplicación puede detenerse temporalmente o debe funcionar de manera continua.
- **Integraciones:** identifica la necesidad de comunicarse con servicios externos, dispositivos, API u otros sistemas empresariales.
- **Seguridad:** determina la sensibilidad de la información y los controles necesarios para protegerla.
- **Presupuesto y tiempo:** condiciona la infraestructura, las herramientas y la complejidad que puede asumir el proyecto.
- **Experiencia del equipo:** permite comprobar si los desarrolladores poseen los conocimientos necesarios para implementar y administrar la arquitectura.
- **Crecimiento esperado:** considera futuros usuarios, módulos, integraciones y cambios en las reglas de negocio.

Procedimiento de selección

La elección puede realizarse mediante un procedimiento de cinco pasos:

1. Identificar las funciones principales y los atributos de calidad requeridos por la aplicación.
2. Clasificar el tamaño del proyecto, la complejidad del dominio, la cantidad de usuarios y el comportamiento de la demanda.
3. Establecer las restricciones de presupuesto, infraestructura, tiempo y conocimientos del equipo.
4. Comparar las ventajas y limitaciones de las arquitecturas disponibles mediante una matriz de criterios.
5. Crear un prototipo y realizar pruebas de rendimiento, integración y mantenimiento antes de aprobar la decisión definitiva.

Este procedimiento evita seleccionar una arquitectura únicamente por preferencia tecnológica. También permite documentar las razones de la decisión y revisarlas cuando la aplicación cambie.

Principales arquitecturas

La arquitectura monolítica integra la interfaz, las reglas de negocio y el acceso a datos en una sola aplicación desplegable. Puede ser adecuada para proyectos pequeños porque facilita el desarrollo, las pruebas y la implementación. IBM indica que el enfoque monolítico continúa siendo una alternativa válida para aplicaciones sencillas (Saraswathi, s. f.).

La arquitectura en capas separa responsabilidades como presentación, lógica de negocio y acceso a datos. Esta organización es utilizada con frecuencia en sistemas empresariales tradicionales porque facilita la distribución de responsabilidades. Sin embargo, un cambio puede atravesar varias capas y afectar diferentes partes de la aplicación (Microsoft, 2025).

La arquitectura de microservicios divide el sistema en servicios pequeños e independientes, organizados alrededor de funciones del negocio. Cada servicio puede desarrollarse, desplegarse y escalarse por separado. Esta independencia puede beneficiar a sistemas grandes y equipos especializados, pero también incrementa la complejidad del monitoreo, la comunicación y la administración de datos. Fowler (2015) advierte que los beneficios de los microservicios deben compararse con los costos propios de un sistema distribuido.

La arquitectura orientada a eventos permite que los componentes reaccionen ante situaciones como una compra, una alerta, una lectura de un sensor o un cambio en la información. Resulta útil en aplicaciones de IoT, notificaciones, procesamiento en tiempo real y sistemas con comunicación asíncrona.

La arquitectura sin servidor o *serverless* permite ejecutar funciones bajo demanda mientras el proveedor de nube administra la infraestructura. Puede utilizarse cuando la carga cambia considerablemente o cuando se procesan eventos específicos. No obstante, deben evaluarse la dependencia del proveedor, los costos acumulados y los límites de ejecución.

Arquitectura según la aplicación

La Tabla 1 presenta una relación general entre algunos tipos de aplicaciones y las arquitecturas que podrían satisfacer sus necesidades. Estas relaciones son recomendaciones y no reglas absolutas.

Cuadro 1: Arquitectura sugerida según el tipo de aplicación

Tipo de aplicación	Arquitectura sugerida	Motivo
Pequeña o académica	Monolítica o en capas	Menor complejidad de desarrollo y despliegue.
Empresarial tradicional	En capas	Separación entre interfaz, reglas de negocio y datos.
Grande y modular	Microservicios	Despliegue y escalamiento independiente de los servicios.
IoT o tiempo real	Orientada a eventos	Procesamiento asíncrono de eventos generados continuamente.
Demanda variable	Sin servidor	Ejecución y escalamiento de funciones bajo demanda.

Fuente: elaboración propia con base en Microsoft (2025), Saraswathi (s. f.) y Fowler (2015).

El tipo de aplicación no determina por sí solo la arquitectura que debe utilizarse. Por ejemplo, una aplicación web pequeña puede funcionar correctamente con una arquitectura monolítica, mientras que una plataforma web con miles de usuarios podría requerir microservicios, procesamiento de eventos y recursos escalables. Por ello, la decisión debe basarse en los requisitos y las características reales del proyecto.

Microsoft (2025) recomienda evaluar si los beneficios de un estilo arquitectónico superan sus dificultades dentro del contexto específico de la aplicación. Esto confirma que no existe una alternativa universalmente superior.

Observaciones y comentarios

La arquitectura más compleja no necesariamente es la más adecuada. En proyectos pequeños, una solución sencilla y correctamente organizada puede ofrecer mejores resultados que una estructura distribuida difícil de administrar. También es importante documentar la decisión y revisarla durante la evolución del sistema. La arquitectura debe resolver necesidades reales, ser comprensible para el equipo y permitir cambios sin aumentar innecesariamente los costos.

Conclusiones

1. La arquitectura debe seleccionarse tomando en cuenta los requisitos funcionales, los atributos de calidad y las restricciones del proyecto.
2. Una aplicación pequeña puede utilizar una arquitectura monolítica o en capas sin asumir innecesariamente la complejidad de un sistema distribuido.
3. No se debe asumir la complejidad de una aplicación sin realizar previamente un análisis de sus usuarios, funciones, presupuesto y crecimiento esperado.
4. Una arquitectura de microservicios puede ser apropiada para aplicaciones complejas que requieren escalabilidad e independencia entre sus servicios.

5. La experiencia del equipo, el presupuesto y las necesidades de mantenimiento también deben evaluarse para seleccionar una arquitectura adecuada para la organización.

Bibliografía

- Fowler, M. (2015, 1 de julio). *Microservice trade-offs*. MartinFowler.com. <https://martinfowler.com/articles/microservice-trade-offs.html>
- Kazman, R., Echeverria, S., & Ivers, J. (2023). *A holistic view of architecture definition, evolution, and analysis* (CMU/SEI-2023-TR-004). Software Engineering Institute, Carnegie Mellon University. <https://doi.org/10.1184/R1/20722402>
- Microsoft. (2025, 14 de octubre). *Architecture styles*. Azure Architecture Center. <https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/>
- Saraswathi, R. (s. f.). *Tipos de arquitecturas de desarrollo de aplicaciones*. IBM. <https://www.ibm.com/es-es/think/topics/application-architecture-types>